

Lista de Exercícios — Aula 05

Revisão de Programação Orientada a Objetos (POO)

VERSÃO PARA OS ESTUDANTES

Prof. Dr. Dilermando Piva Jr. | 2º semestre — Ciência de Dados para Negócios

Conteúdos mobilizados

- Paradigma orientado a objetos; relação entre classe e objeto; atributos e métodos.
- Pilares da POO: abstração, encapsulamento, herança e polimorfismo.
- Definição e instanciação de classes em Python; `__init__`, `self` e `__str__`.
- Atributos privados/protegidos, getters e setters; interface do objeto.
- Atributos de classe (estáticos) e troca de mensagens entre objetos.

Orientações

- Leia o enunciado inteiro antes de programar.
- Nos exercícios de programação, use Python e mantenha nomes de classes em **CamelCase** e nomes de atributos/métodos em **snake_case**.
- Teste cada classe com objetos criados no programa principal.
- Quando houver encapsulamento, evite alterar diretamente atributos que deveriam ser privados.
- Soluções equivalentes e corretamente justificadas também podem ser consideradas.

Parte A — Questões de múltipla escolha

Exercício 1 — Objetos, encapsulamento e interface [Médio-difícil]

Considere uma classe `ContaBancaria` em que o atributo `__saldo` é privado e as operações `depositar()`, `sacar()` e `consultar_saldo()` são públicas. Qual alternativa interpreta melhor essa modelagem segundo os princípios estudados?

- A)** O atributo `__saldo` é abstrato, pois não existe fisicamente no mundo real.
- B)** O encapsulamento permite ocultar a representação interna do saldo e disponibilizar uma interface de alto nível para manipulação do objeto.
- C)** A herança impede que o saldo seja modificado diretamente por outras classes.
- D)** O polimorfismo exige que `depositar()` e `sacar()` tenham exatamente o mesmo código em todas as subclasses.
- E)** O uso de `__saldo` torna impossível qualquer alteração do saldo, inclusive pelos métodos da própria classe.

Exercício 2 — Herança e polimorfismo [Médio-difícil]

Uma aplicação possui a classe `Funcionario`, com o método `calcular_bonus()`. As classes `Vendedor` e `Gerente` herdam de `Funcionario` e redefinem `calcular_bonus()` de formas diferentes. Ao percorrer uma lista de objetos e executar `funcionario.calcular_bonus()`, cada objeto produz o resultado adequado ao seu tipo. Quais conceitos aparecem de forma mais direta?

- A) Somente abstração.
- B) Somente encapsulamento.
- C) Herança e polimorfismo.
- D) Encapsulamento e atributo de classe.
- E) Apenas atributo privado e getter.

Exercício 3 — Atributo de instância x atributo de classe [Médio-difícil]

Observe o código: `class Produto: contador = 0; def __init__(self, nome): Produto.contador += 1; self.nome = nome`. Depois são criados três objetos `Produto`. Qual afirmação é correta?

- A) Cada objeto terá seu próprio contador iniciado em zero.
- B) `Produto.contador` será compartilhado e terá valor 3 após as três instanciações.
- C) `self.nome` será compartilhado entre todos os objetos.
- D) contador só poderia ser acessado por um método getter, pois todo atributo de classe é privado.
- E) O uso de contador transforma `Produto` automaticamente em uma superclasse.

Parte B — Programação: nível fácil

Exercício 4 — Classe Livro: atributos e métodos [Fácil]

Crie uma classe `Livro` para representar um livro de uma biblioteca.

Requisitos:

- O construtor deve receber título, autor e páginas.
- Crie o método `descricao()`, que retorne uma string no formato: “Título — Autor (N páginas)”.
- Instancie dois livros diferentes e exiba suas descrições.

Exercício 5 — Conta simples com encapsulamento [Fácil]

Crie uma classe `Conta` que mantenha o saldo protegido contra alteração direta.

Requisitos:

- O construtor recebe o nome do titular e inicia o saldo em 0.
- Use `__saldo` como atributo privado.
- Implemente `depositar(valor)`, permitindo apenas valores positivos.
- Implemente `consultar_saldo()`, retornando o saldo atual.
- Crie uma conta, faça dois depósitos e mostre o saldo.

Exercício 6 — Animais: herança e polimorfismo [Fácil]

Implemente uma pequena hierarquia de animais para exercitar herança e polimorfismo.

Requisitos:

- Crie a classe Animal com atributo nome e método emitir_som().
- Crie Cachorro e Gato herdando de Animal.
- Redefina emitir_som(): Cachorro retorna “au au” e Gato retorna “miau”.
- Crie uma lista com um cachorro e um gato e percorra-a chamando emitir_som() sem testar o tipo do objeto.

Parte C — Programação: nível médio

Exercício 7 — Veículo com velocidade controlada [Média]

Modele um veículo inspirado no exemplo Automovel da aula, mas com regras completas de aceleração e frenagem.

Requisitos:

- A classe Veiculo deve ter placa, velocidade_max e velocidade_atual; torne os três atributos privados.
- velocidade_atual começa em 0.
- Crie getters para placa e velocidade_max.
- Crie um setter para velocidade_max que aceite apenas valores positivos e nunca deixe a nova máxima menor que a velocidade atual.
- acelerar(incremento=10) aumenta a velocidade sem ultrapassar a máxima.
- frear(decremento=10) reduz a velocidade sem ficar negativa.
- Implemente __str__ para mostrar placa, velocidade atual e máxima.
- Demonstre o funcionamento com várias acelerações e frenagens.

Exercício 8 — Funcionários e cálculo de bônus [Média]

Crie uma hierarquia para calcular bônus de funcionários de uma empresa.

Requisitos:

- Funcionario possui nome e salario; implemente __str__.
- Funcionario.calcular_bonus() retorna 5% do salário.
- Vendedor herda de Funcionario, acrescenta vendas e redefine calcular_bonus() para 5% do salário + 2% das vendas.
- Gerente herda de Funcionario, acrescenta equipe e redefine calcular_bonus() para 10% do salário + R\$ 100 por integrante da equipe.
- Crie pelo menos um objeto de cada tipo, coloque-os em uma única lista e exiba nome, tipo e bônus usando o mesmo método calcular_bonus().

Exercício 9 — Produtos e contador de objetos [Média]

Crie uma classe Produto para um catálogo de e-commerce, explorando atributo de classe e encapsulamento.

Requisitos:

- Crie o atributo de classe contador iniciado em 0.
- Cada produto deve receber automaticamente um id sequencial a partir do contador.
- Use atributos privados para id, nome e preco.
- Crie getters para id, nome e preco.

- Crie `set_preco(novo_preco)`, aceitando apenas valores maiores que zero.
- Implemente `aplicar_desconto(percentual)`, validando `percentual` entre 0 e 100.
- Implemente `__str__`.
- Crie três produtos e, ao final, mostre quantos objetos foram criados usando `Produto.contador`.

Parte D — Desafio integrador

Exercício 10 — Mini sistema de pedidos de e-commerce [Difícil]

Desenvolva um pequeno sistema orientado a objetos para pedidos de um e-commerce. O objetivo é integrar a maior parte dos conceitos trabalhados na aula em um único problema.

Requisitos:

- Crie uma classe `Produto` com `id` automático (atributo de classe), `nome` e `preço` privados; inclua `getters`, `setter` validado para `preço` e `__str__`.
- Crie `ProdutoFisico` herdando de `Produto`, com `peso`, e `ProdutoDigital` herdando de `Produto`, com `tamanho_mb`.
- Em `Produto`, crie `calcular_frete()` retornando 0. Redefina-o em `ProdutoFisico` para retornar R\$ 5,00 + R\$ 2,00 por kg. `ProdutoDigital` deve manter `frete` igual a 0. Essa chamada comum deve demonstrar polimorfismo.
- Crie uma classe `Cliente` com `nome` e `email` privados, `getters` e `__str__`.
- Crie uma classe `Pedido` com `número` automático (atributo de classe), `cliente`, `lista de produtos` e `status` privado.
- `Pedido` deve oferecer `adicionar_produto(produto)`, `calcular_total()`, `calcular_frete_total()`, `fechar_pedido()` e `__str__`.
- Depois de fechado, o pedido não deve aceitar novos produtos. Essa regra deve ser controlada internamente pela classe.
- `calcular_total()` deve somar os preços dos produtos. `calcular_frete_total()` deve chamar `calcular_frete()` de cada item sem testar se ele é físico ou digital.
- No programa principal, crie um cliente, ao menos três produtos (misturando físicos e digitais), monte um pedido, exiba o resumo, feche-o e tente adicionar mais um produto para comprovar a regra.

Checklist antes da entrega

- O programa executa sem erros?
- As classes têm responsabilidades claras?
- Os atributos privados são acessados pela interface apropriada?
- Os métodos respeitam as regras do enunciado?
- Nos exercícios com herança, a relação “é um(a)” faz sentido?
- O polimorfismo é demonstrado chamando o mesmo método em objetos diferentes?

A entrega deve ser feita diretamente no repositório GitHub.